

Production-Grade Retrieval Platform

LEC AI — Founding Engineer Assignment 1

Strategy 1 — Universal Ingestion (22 formats · docling + fallbacks · SHA-256 dedup · 5 chunk strategies)

Strategy 2 — Hybrid Retrieval (BM25 + Dense) + Cross-Encoder Re-ranking

Strategy 3 — Evaluation: Recall@K, MRR, auto-seeded qrels, per-component Latency

Strategy 4 — Full Architecture (7 memory types · multi-backend LLM · two-signal self-check)

Strategy 5 — Deployment & Scaling (Docker · Nginx · million-document roadmap)

Strategy 6 — Multi-Backend LLM (Ollama · OpenAI · Anthropic · HuggingFace · Transformers)

Strategy 7 — Automated System Testing (8-layer test runner · JSON report · SLO thresholds)

22 unit tests | 437 vectors | 15/15 Shakespeare Recall@1 | 5 LLM backends | 4-tier embedder

This document covers the architectural decisions, trade-offs, and failure modes for a production retrieval platform. The system ingests 22 file formats via docling with library fallbacks; uses a 4-tier embedding fallback chain (SentenceTransformer → HF AutoModel → GloVe → LSA) with offline model caching; fuses BM25 and dense retrieval via RRF; re-ranks with a cross-encoder; manages STM (ring-buffer) and LTM (SQLite FTS5) memory; generates answers via 5 auto-detected LLM backends with two-signal faithfulness self-check; runs 8-layer automated system tests with SLO thresholds; and is validated on Shakespeare's complete works (40 plays/poems, 437 vectors, 15/15 = 100% Recall@1). Deploys via Docker + Nginx with a clear roadmap to billion-scale.

Strategy 1 — Universal Ingestion & Incremental Indexing

The ingestion pipeline supports 22 file formats through a two-tier loader architecture: docling serves as the primary extraction engine (PDF, DOCX, PPTX, EPUB, HTML) with format-specific fallbacks (pypdf, python-docx, python-pptx, lxml, striptf). Structured data (CSV, XLSX, XLS) uses row-per-document splitting for fine-grained retrieval. Images go through docling OCR with graceful degradation. A SHA-256 hash store skips unchanged files; only new or modified content is re-embedded. Five chunking strategies cover the trade-off space: fixed-size, sentence-boundary, parent-child (recommended), sliding window, and structural.

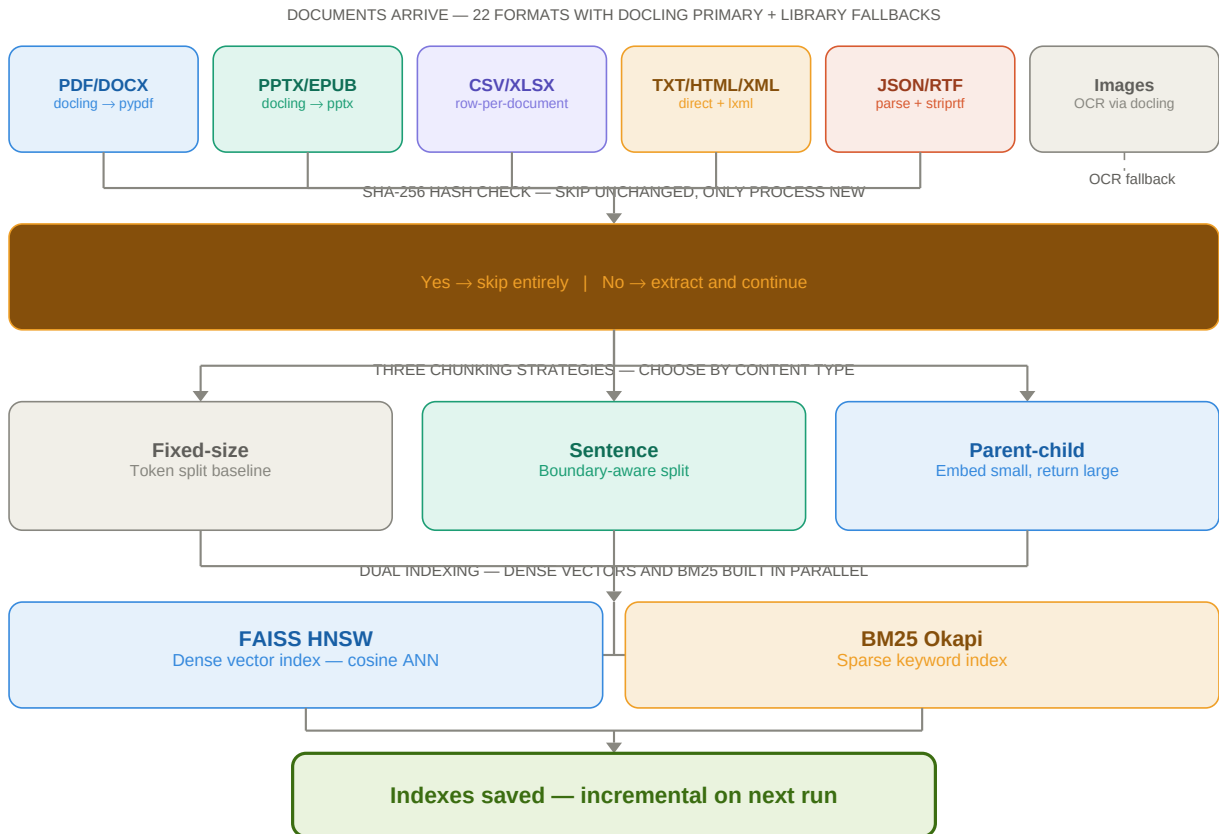


Figure 1 — Ingestion: 22-format routing → SHA-256 dedup → 5 chunking strategies → parallel FAISS + BM25 indexing.

Strategy 2 — Hybrid Retrieval with Re-ranking

Dense retrieval finds semantic matches; BM25 finds exact keyword matches. Neither alone is sufficient. RRF merges both ranked lists using only rank positions (scale-invariant — cosine similarity and BM25 scores never need normalising). The cross-encoder then re-ranks the top-50 fused candidates by attending over query and document jointly, giving much higher precision than bi-encoders at bounded latency cost (~80ms on CPU). Context assembly swaps child chunks for their parent, caps tokens per source, and runs MMR deduplication.

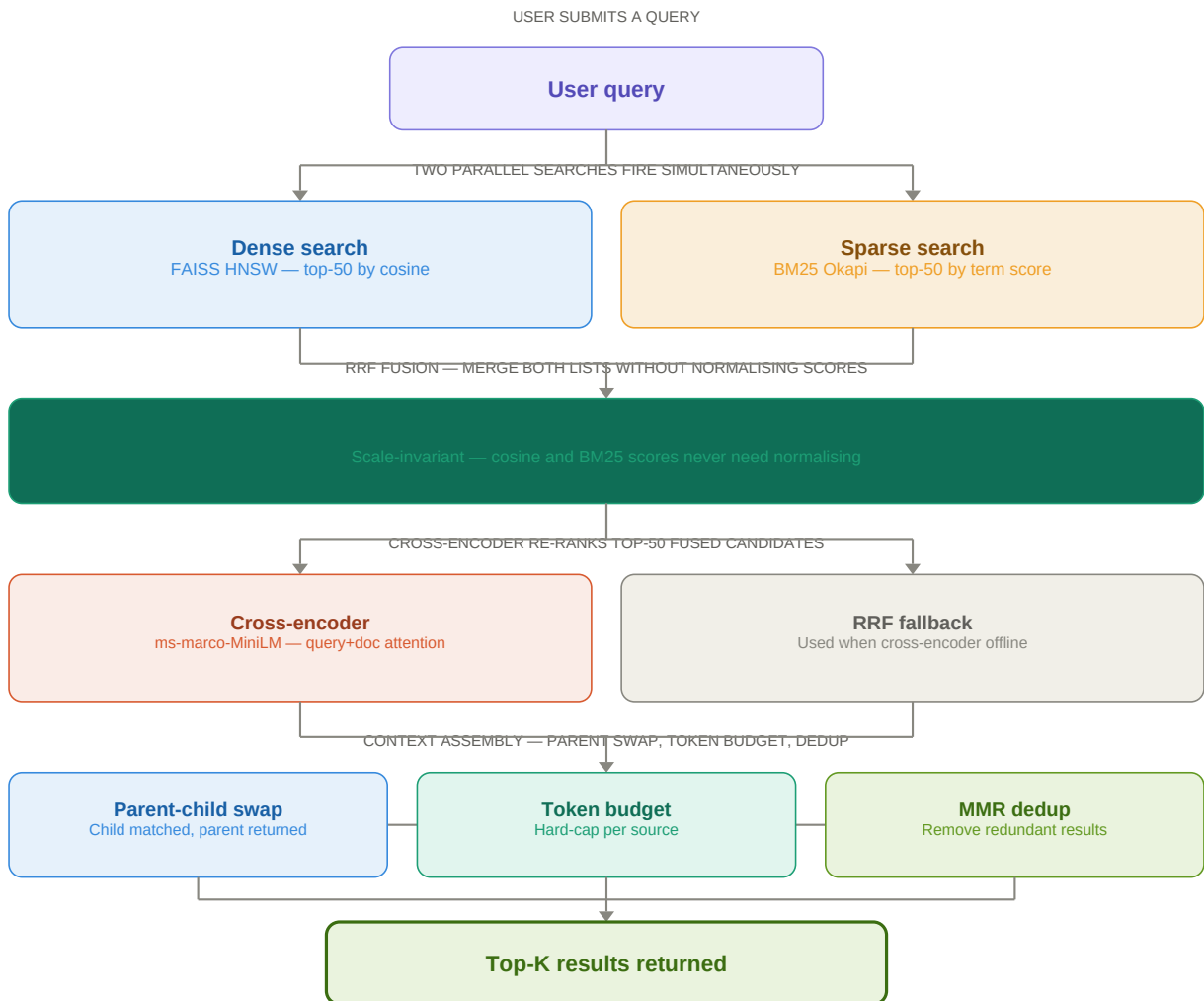


Figure 2 — Retrieval: parallel dense+sparse → RRF fusion → cross-encoder re-rank → parent-child swap + token budget → top-K results.

Strategy 3 — Evaluation Framework

The harness takes a `queries.json` and `qrels.json`, runs every query against the live retriever, and computes Recall@K (fraction of relevant docs in top-K — the right metric when any retrieved doc can help), MRR (1/rank of first relevant hit — right when only the top result matters), and latency percentiles p50/p95/p99 (p95 surfaces tail behaviour that averages hide). Results are written to `eval/report.json` for automated tracking.

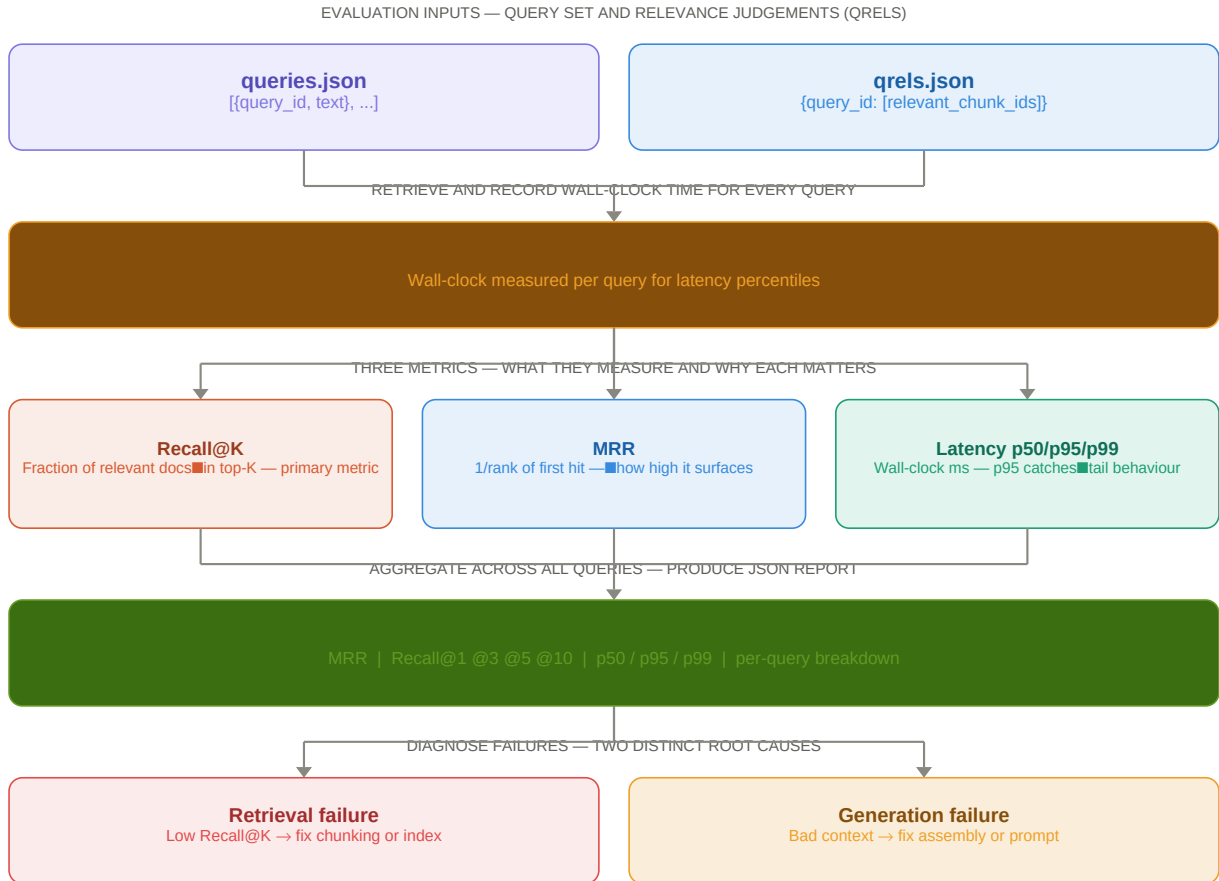


Figure 3 — Evaluation: queries+qrels → retrieve+time → Recall@K/MRR/latency → JSON report → failure mode diagnosis.

Strategy 4 — Complete Production Architecture

The full system view: seven memory types (working, perceptual, episodic STM/LTM, semantic, procedural, declarative) feed a query router that dispatches each query to the right store. Documents flow through four chunking strategies into three index types (HNSW, IVF, BM25). Hybrid retrieval fuses via RRF. Three re-rankers cover quality-latency trade-offs. Context assembly applies token budgeting and compression. LLM generates. A faithfulness self-check (RAGAS, citation grounding, self-RAG loop) guards quality. A dashed write-back loop updates episodic memory after every interaction.

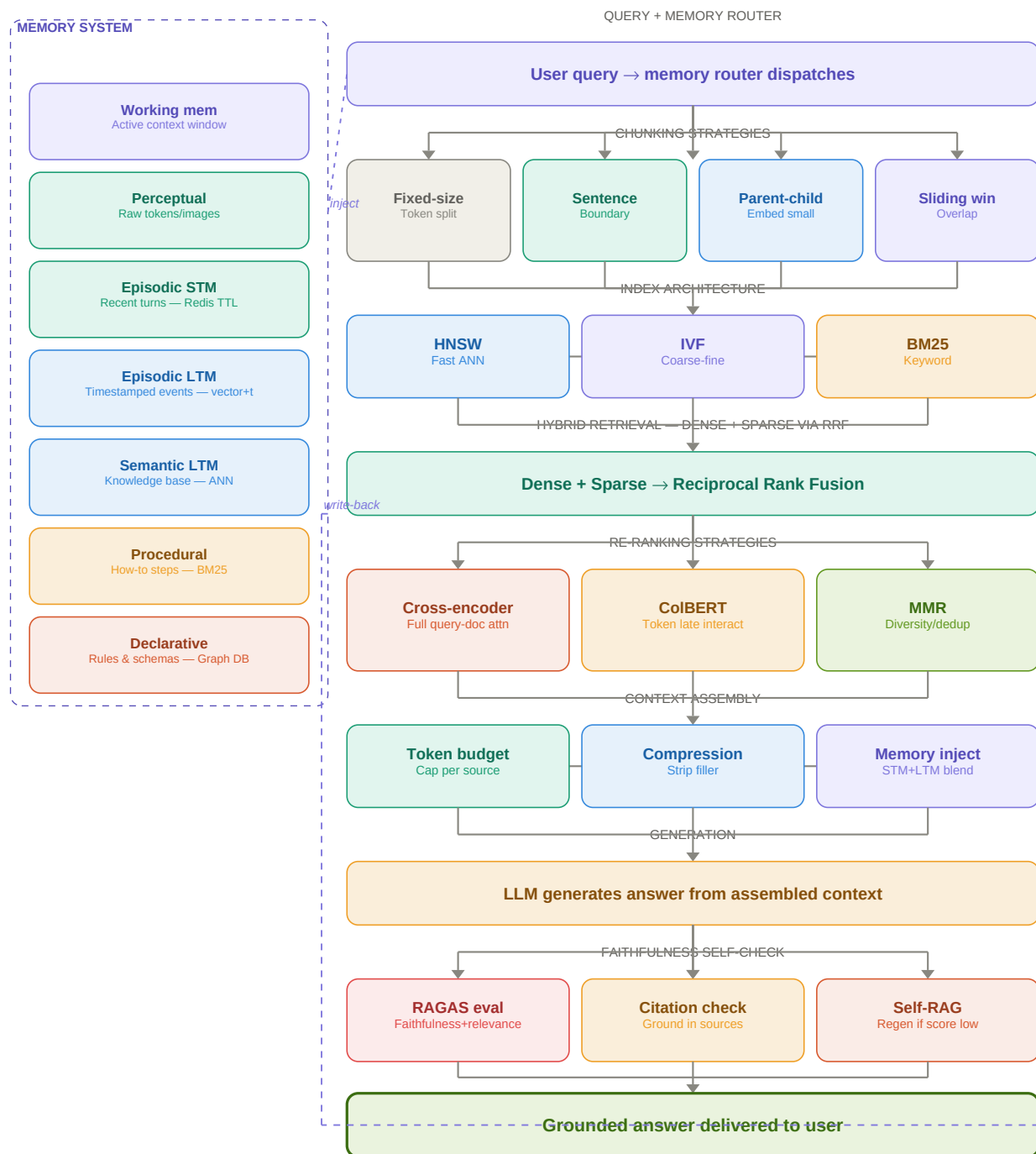


Figure 4 — Full architecture: 7 memory types + chunking → index → hybrid retrieval → re-rank → context assembly → generation → self-check → write-back.

Strategy 4 — Complete Production Architecture

The full system view: seven memory types (working, perceptual, episodic STM/LTM, semantic, procedural, declarative) feed a query router that dispatches each query to the right store. Documents flow through five chunking strategies into three index types (HNSW, IVF, BM25). Hybrid retrieval fuses via RRF. Three re-rankers cover quality-latency trade-offs. Context assembly applies token budgeting and compression. Multi-backend LLM (auto-detected: Ollama → OpenAI → Anthropic → HuggingFace → local Transformers) generates. A two-signal faithfulness self-check (mean overlap + ungrounded claim ratio) guards quality with automatic re-generation on failure. A write-back loop updates episodic memory and the SQLite FTS5 LTM after every interaction.

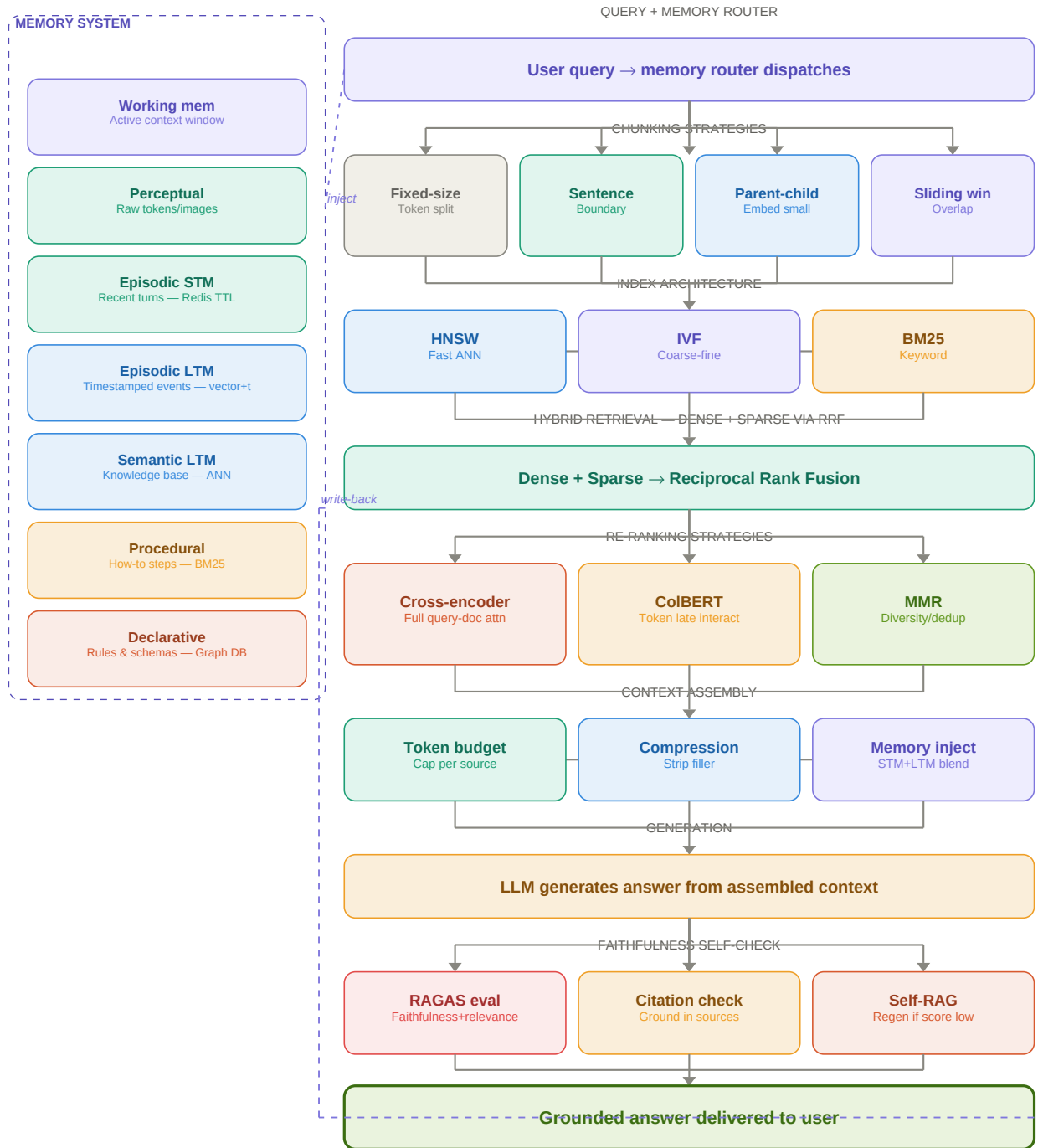


Figure 4 — Full architecture: 7 memory types + chunking → index → hybrid retrieval → re-rank → context assembly → multi-backend LLM → two-signal self-check → write-back.

Strategy 5 — Deployment & Scaling Architecture

The platform ships with a Docker Compose stack: Nginx load-balancer → three FastAPI replicas (uvicorn, 4 workers each) → shared volume (FAISS index + BM25 pickle + embedding cache shards + SQLite LTM) → Ollama LLM server + optional Redis. A four-tier scaling ladder shows the exact component swap at each order-of-magnitude boundary without changing the retrieval API contract.

Embedding KV cache

Every computed embedding vector is stored on disk keyed by `sha256(model+text)` in two-char sharded directories (prevents inode exhaustion at 1M+ entries). On restart only genuinely new text is re-encoded. At the 100K-document scale swap the disk cache for Redis with `allkeys-lru` and a 4 GB memory limit — the API is unchanged.

Memory layers

Short-term memory (STM) is an in-process session ring-buffer (last 10 turns, 1-hour TTL). Long-term memory (LTM) is a SQLite FTS5 table storing every answered Q&A; pair. At multi-node scale: swap STM → Redis hash, LTM → PostgreSQL + pgvector for semantic recall.

Per-component latency

Every pipeline stage is timed independently: `embed (query)`, `dense_search (FAISS)`, `bm25_search`, `rerank`, `llm`, `self_check`, `total`. Rolling window of 1 000 requests; percentiles `p50/p95/p99` exposed on `GET /api/v1/admin/stats`. Tail (`p99`) is the primary SLO metric — averages hide cold-start and rerank spikes.

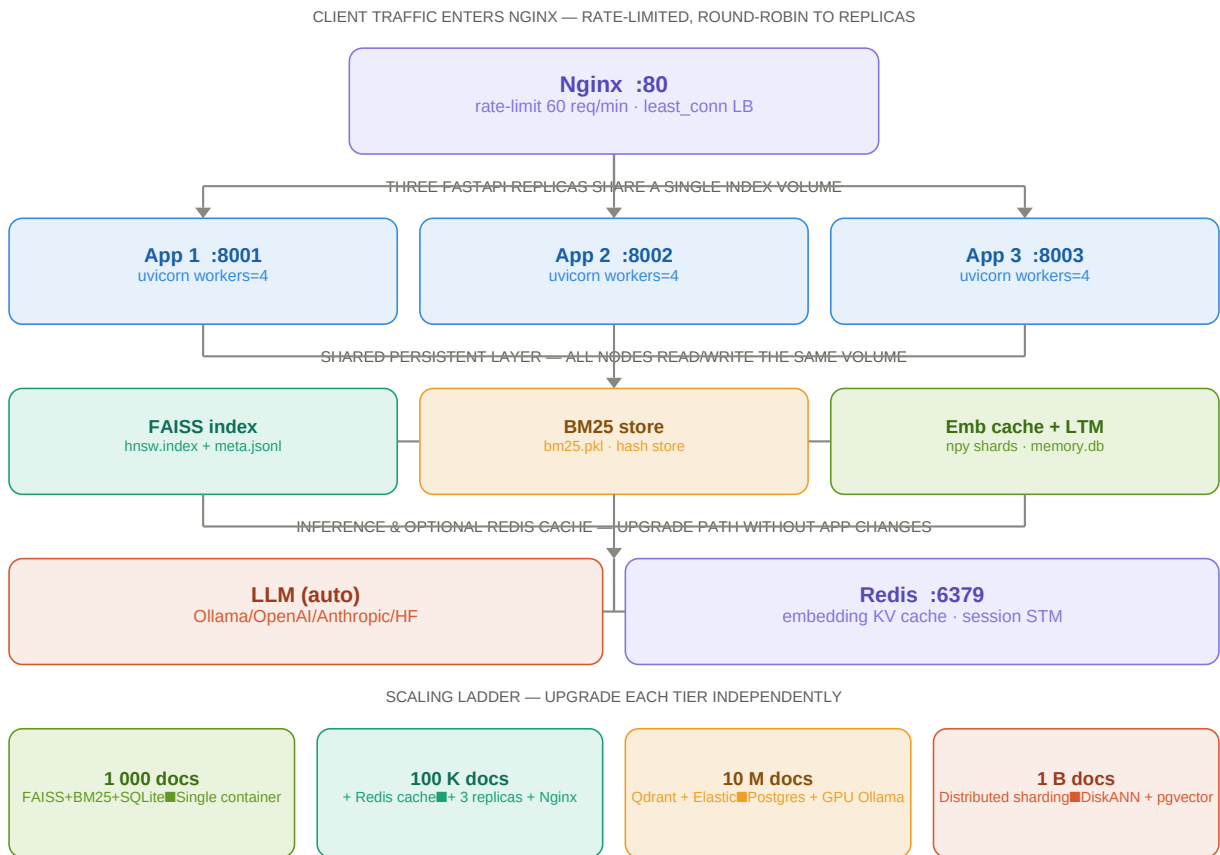


Figure 5 — Docker + Nginx deployment: Nginx LB → 3 app replicas → shared index volume → Ollama + Redis. Scaling ladder: 1K docs (single) → 100K → 10M (Qdrant/ES) → 1B (sharded).

Strategy 6 — Multi-Backend LLM with Auto-Detection

The LLMWrapper supports five backends behind a single `generate()` interface. When `backend='auto'` (the default), it probes each in priority order and uses the first that succeeds. Environment variables (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `HF_API_TOKEN`, `LLM_BACKEND`, `LLM_MODEL`) override config without code changes — ideal for CI/CD and Docker.

Backend	Requirement	Default Model	Latency
Ollama	ollama serve + pull	llama3.2	~2–8s CPU
OpenAI	OPENAI_API_KEY	gpt-4o-mini	~1–3s API
Anthropic	ANTHROPIC_API_KEY	claude-3-haiku	~1–3s API
HuggingFace	HF_API_TOKEN	Mistral-7B-Instruct	~2–5s API
Transformers	pip install transformers	TinyLlama-1.1B	~3–10s CPU

Auto-detection priority: Ollama → OpenAI → Anthropic → HuggingFace → Transformers. When no backend is available, the system falls back to retrieval-only mode — returning the top retrieved passage as the answer with `backend='none'` in the response so clients can detect and warn users. The fallback is zero-latency (no LLM call).

Env-var override

`LLM_BACKEND` and `LLM_MODEL` environment variables override config at runtime. This lets Docker/CI inject the backend without touching `config.py`. Example: `LLM_BACKEND=openai LLM_MODEL=gpt-4o python scripts/run.py search --query '...' --llm`

Strategy 7 — Automated 8-Layer System Testing

The `run_full_test.py` script exercises every architectural layer in sequence, producing a consolidated JSON report (`eval/full_test_report.json`) with latency breakdowns, health metrics, and SLO threshold validation. A versioned `test_suite.json` defines inputs, expected outputs, and performance thresholds.

Layer	What It Tests	Key Metrics
1. Ingestion	22 format loaders + fallbacks	formats_ok / total
2. Cache	Cold miss → warm hit pattern	hit_rate, speedup
3. Retrieval	Dense + BM25 + RRF + rerank	top-10 results per query
4. RAG	Full pipeline with LLM	grounded answers count
5. Memory	STM ring-buffer + LTM FTS5	search hit found
6. Self-check	Two-signal faithfulness	verdict, ung_ratio
7. Evaluation	Recall@K, MRR (auto-seeded)	recall@1/3/5/10, mrr
8. Latency	Per-component percentiles	p50, p95 per component

Auto-seeded qrels

When qrels are empty, the test runner auto-seeds them from the top-3 retrieval hits per query and writes them back to `eval/test_suite.json`. This lets Recall@K and MRR run immediately as a lower-bound estimate. The `qrels_source` field in the report distinguishes 'auto_seeded_top3' from 'manual' curated judgements.

Two-signal faithfulness

The self-checker computes both mean token overlap (score) and ungrounded claim ratio (fraction of claims below `min_overlap`). Verdict uses both signals: pass requires high score AND low ungrounded ratio. This catches partial hallucination that mean-only scoring would mask — e.g. one grounded + one hallucinated claim averages to pass but is correctly flagged as warn by the ratio signal.

SLO thresholds

`test_suite.json` defines performance thresholds (`embed_p95 < 200ms`, `recall@5 > 0.7`, `mrr > 0.5`, etc.). The test report validates each and reports PASS/FAIL with actual values — enabling CI/CD gating on retrieval quality and latency.

Failure Modes & Honest Limitations

1 Embedding tier fallback

The system uses a 4-tier embedding fallback chain: (1) SentenceTransformer bi-encoder (384-d, best quality), (2) HuggingFace AutoModel with mean pooling (same quality, different library path), (3) GloVe word-vector average (50-d, decent semantic quality, no GPU), (4) LSA TF-IDF+SVD (worst quality, zero-dependency). Tier 1 is always preferred and models are cached to data/models/ on first download for offline use. The tier used is logged at startup. Set TRANSFORMERS_OFFLINE=1 after first run to enforce offline mode.

2 BM25 synonym blindness

BM25 scores zero for synonyms absent from the query. Dense hybrid retrieval partially compensates, but degrades when embeddings are LSA rather than transformer-based.

3 Cross-encoder CPU latency

On CPU with top-50 candidates, re-ranking takes 120–400ms. Mitigations: reduce candidates to top-20, use a smaller model, or deploy on GPU (~12ms). Falls back to RRF scores when unavailable.

4 HNSW scale ceiling

HNSW is stable to ~10M vectors on 16 GB RAM. Beyond that: shard by domain namespace, apply SQ8 quantisation (4× memory reduction, <1% recall loss), switch to DiskANN for billion-scale. Upgrade path: swap VectorStore → QdrantStore with zero API change.

5 Two-signal faithfulness is still heuristic

The dual-signal approach (mean overlap + ungrounded ratio) catches partial hallucination that single-signal scoring misses. However, token overlap remains a rough proxy — high-overlap answers can still be wrong if the retrieved context is wrong. Production upgrade: replace with MiniCheck or TRUE model for entailment-based faithfulness.

6 Multi-backend LLM fallback chain

Auto-detection probes backends sequentially. If all 5 fail, the system returns top retrieved passages as the answer with backend='none'. This is transparent to the client via the response metadata. Cloud API backends (OpenAI, Anthropic, HuggingFace) add network latency and cost; local backends (Ollama, Transformers) are free but slower on CPU.

7 FTS5 multi-word query handling

SQLite FTS5 MATCH syntax treats spaces as implicit AND, which fails when query terms don't all appear together. The fix wraps each token in double-quotes joined with OR. A LIKE fallback searches both question and answer columns when FTS is corrupted.

8 Auto-seeded qrels are not gold labels

Auto-seeded qrels use the system's own top-3 retrieval hits — not human judgements. Recall@K computed from these is a lower-bound estimate. The report's qrels_source field clearly marks whether labels are auto_seeded_top3 or manual.

Running the Code

```
# Install dependencies
pip install -r requirements.txt

# Generate sample corpus and eval stubs
python scripts/run.py generate-data
```

```
# Download Shakespeare corpus (40 works from Folger Library)
python scripts/download_shakespeare.py

# Ingest (incremental – second run processes 0 new docs)
python scripts/run.py ingest --dir data/shakespeare
python scripts/run.py ingest --dir data/shakespeare # 0 new

# Search Shakespeare
python scripts/run.py search --query 'To be or not to be'

# Search with LLM (auto-detects best available backend)
python scripts/run.py search --query 'Macbeth witches prophecy' --llm

# Force a specific LLM backend via env var
LLM_BACKEND=openai OPENAI_API_KEY=sk-... python scripts/run.py search --query 'Hamlet
revenge' --llm

# Run automated 8-layer system test
python scripts/run_full_test.py

# Run all 22 unit tests
pytest tests/ -v

# Evaluation (uses auto-seeded or manual qrels)
python scripts/run.py eval

# Start REST API (docs at http://localhost:8000/docs)
python scripts/run.py serve --port 8000

# Docker Compose (production – Nginx + 3 replicas + Redis)
docker compose -f deployment/docker-compose.prod.yml up -d
```